Code559

Front-End Developer Course

MODULE 11

Debugging and Testing

Read the red line, find the cause, change one thing, and prove it works with a written test.

Free lesson: 559solutions.com/code559/module-11-debugging-testing/

In this packet

1. **The lesson**
2. **Project: A test plan for every interactive feature on your site**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn to read errors, track down bugs with the Console, test with the keyboard, and write Action / Expected / Actual test plans. It turns fixing into a method, not luck: guess-and-edit stops working once a page has real JavaScript, and a written test can be checked again after every change.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. What is a bug, exactly? 5
  - 2. How do you read a Console error? 5
  - 3. What do the three common error types usually mean? 6
  - 4. What does `null` mean, and why does a `TypeError` follow it? 7
  - 5. How do you debug a selector? 7
  - 6. How do you see what a variable holds? 8
  - 7. Why change one thing at a time? 9
  - 8. What is the browser debugger? 9
  - 9. How do you test with the keyboard? 10
  - 10. What is the Action / Expected / Actual test matrix? 11
  - 11. What is an edge case? 12
- Exercise 12
- Common misconception 17
- Summary 17
- Check your understanding 17
- Before next module 18
- Go further 18
- What to keep 18

Part 2: Project: A test plan for every interactive feature on your site 19

Part 3: Finishing work in Git 27

Part 4: Troubleshooting: five common problems 30

Part 5: Self-assessment 33

Part 6: Answers 35

About 559 Solutions 41

© 2026 559 Solutions LLC

PART 1

The lesson

Read the red line, find the cause, change one thing, and prove it works with a written test.

Time: about 2 hours (estimate)

In this module you will learn:

- What a bug is in plain terms: a gap between what you expected and what happened
- How to read a Console error: its type, its message, and the file name and line number
- Three common error types (`SyntaxError`, `ReferenceError`, `TypeError`) and what each one usually means
- What `null` means when it comes back from `document.querySelector()`, and the `TypeError` that follows it
- How to debug a selector by testing it in the Console
- How to use `console.log()` to see what a variable holds at the moment it matters
- Why you change one thing at a time
- What the browser debugger and a breakpoint are (awareness only)
- How to test a page with the keyboard alone, and what a focus state looks like
- The **Action / Expected / Actual** test matrix, the testing standard for the rest of this course
- How to test edge cases: empty input, the first item, the last item, and wrap-around

---

Why this matters

Every page you have built so far has broken at least once. Up to now you fixed things by looking hard and trying edits until something worked. That works on small pages. It stops working the day a page has forty lines of JavaScript and the button does nothing.

Debugging is a method, not a talent. You say what you expected, you look at what happened, and you find the first place where the two split. The Console tells you more than most beginners read. A short `console.log()` line tells you the rest.

Testing is the other half. A test is a written promise: "when I do this, I expect that." If you write the promise down, you can check it again tomorrow, after every change. From this module on, every interactive feature you build gets a small written test plan in one shared format.

Before you start

1. Finish Module 10. You should be able to make a page respond to a click or a form submit with `addEventListener()`.
2. In your course folder (for example `Documents\code559`), create a throwaway folder called `practice-11`. Nothing in it is published. Open it in VS Code with **File → Open Folder...**.

© 2026 559 Solutions LLC

3. Open Edge or Chrome. You will press **F12** often in this module. Click the **Console** tab once so it opens there next time.
4. Have paper or a blank text file ready. You will write predictions before you run things.

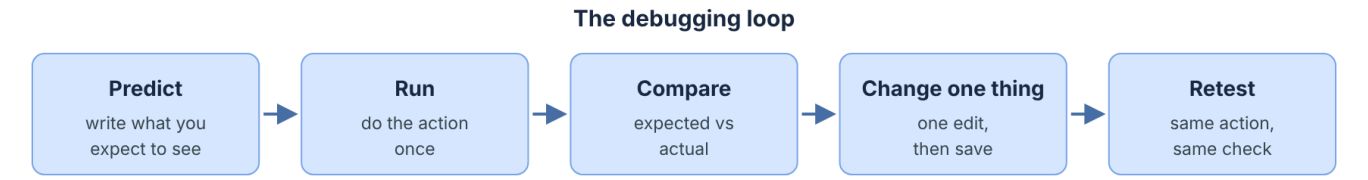
> **Checkpoint.** You have an empty `practice-11` folder open in VS Code, and you can open the Console with F12.

Core concepts

1. What is a bug, exactly?

A bug is a difference between what you **expected** and what **actually** happened. Nothing more. If you cannot say what you expected, you cannot say there is a bug.

MDN's *What went wrong? Troubleshooting JavaScript* sorts errors into kinds. A **syntax error** is a spelling or punctuation mistake that stops the code from running. A **logic error** is code that runs but gives the wrong result; MDN notes these are harder, because "there usually isn't an error message to direct you to the source of the error." So the first habit is to write down your expectation before you click. Then the comparison is easy.



The loop you will use for every bug in this module. Notice that "Compare" needs a written prediction to compare against.

Open your `practice-10` page from Module 10 (or your own site, if you did the packet projects) and pick one button. Before you click it, write one sentence: "When I click ___, I expect ___." Click it. Did the actual result match?

2. How do you read a Console error?

When JavaScript cannot go on, it stops and prints a red line in the Console. That line has three parts, and each one is a clue.

© 2026 559 Solutions LLC

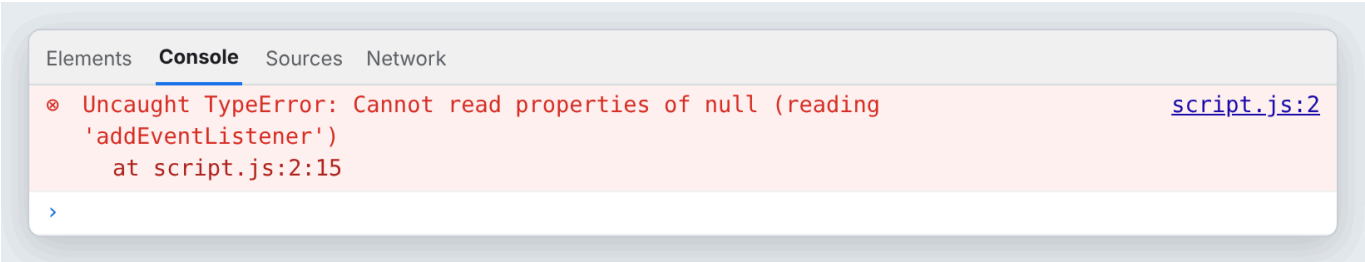
Here is a short script for a page with a button whose id is `message-button` and a paragraph whose id is `message`. The id in the script is misspelled:

```
const messageButton = document.querySelector("#mesage-button");
messageButton.addEventListener("click", showMessage);

function showMessage() {
  document.querySelector("#message").textContent = "Hello!";
}
```

The function is written below line 2, which uses it. That is allowed: MDN's *function* page says "You can use the function before you declared it." It is placed there only so the error stays on line 2; in your own scripts, keep Module 9's order (find, define, connect).

Read the red line from left to right. **The type** comes first (`TypeError`): what kind of problem. **The message** comes next: what JavaScript tried to do. **The location** is on the right (`script.js:2`): the file and the line number. Click that link in DevTools and it opens the file at that line. The `at script.js:2:15` line underneath adds the column: character 15 on line 2.



Type, message, location. The message says "of null": the variable on line 2 holds `null`. (real output, rendered; the error text, file name and line came from Chromium loading this page from a file. The Console frame is drawn.) Your Console may look slightly different: if you do not see the `at …` lines, click the small arrow beside the red message to open them.

Find the three parts of that error and write them down: type, message, file and line.

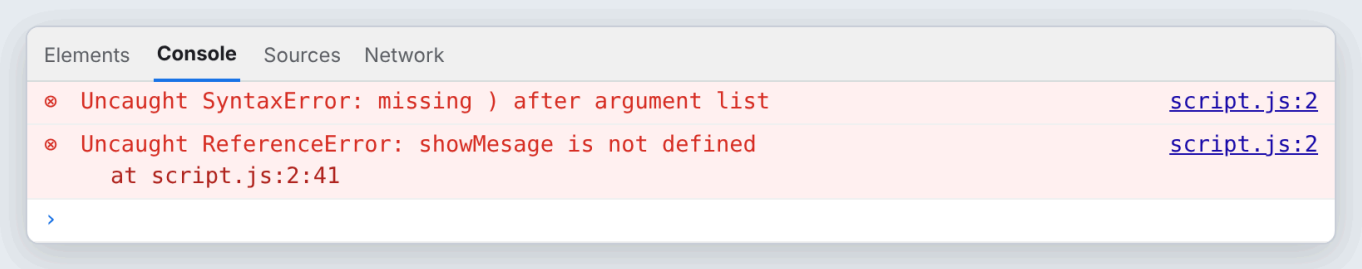
3. What do the three common error types usually mean?

You will see three error types far more than any others.

A **`SyntaxError`** means the code is not valid JavaScript, so none of the file runs; look for a missing bracket, quote or parenthesis. A **`ReferenceError`** means you used a name that does not exist; it is almost always a typo or a variable used before it was made. A **`TypeError`** means the name exists, but its value is the wrong kind for what you asked; the most common one in front-end work is trying to use `null` as if it were an element.

© 2026 559 Solutions LLC

The same script, with the id spelled correctly, broken two other ways. First a missing `)` at the end of line 2, then a typo in the function name (`showMesage`):



Both errors point at line 2, but they mean different things. The `SyntaxError` has no `at …` line because the file never started running. (real output, rendered; captured in Chromium from two pages loaded as files. The Console frame is drawn.)

Without running anything: which type would you get from `consol.log("hi")`? (Answer: `ReferenceError`, because `consol` is not a name JavaScript knows.)

4. What does `null` mean, and why does a `TypeError` follow it?

`document.querySelector()` looks for the first element that matches a CSS selector. MDN's page *Document: querySelector() method* says it plainly: "If no matches are found, `null` is returned."

So `null` here means **JavaScript did not find the element**. It is not an error yet. The error comes on the next line that uses the variable, because `null` has no `addEventListener` and no `textContent`. That is why the red line often points at a different line from the real mistake. MDN's troubleshooting page warns about exactly this: "the error will not necessarily be on that line." The line number tells you where `null` was *used*. You still have to trace back to where it was *made*.

When you see "of null" in a message, go to the `querySelector` line that filled that variable, and compare its selector with the HTML, letter by letter.

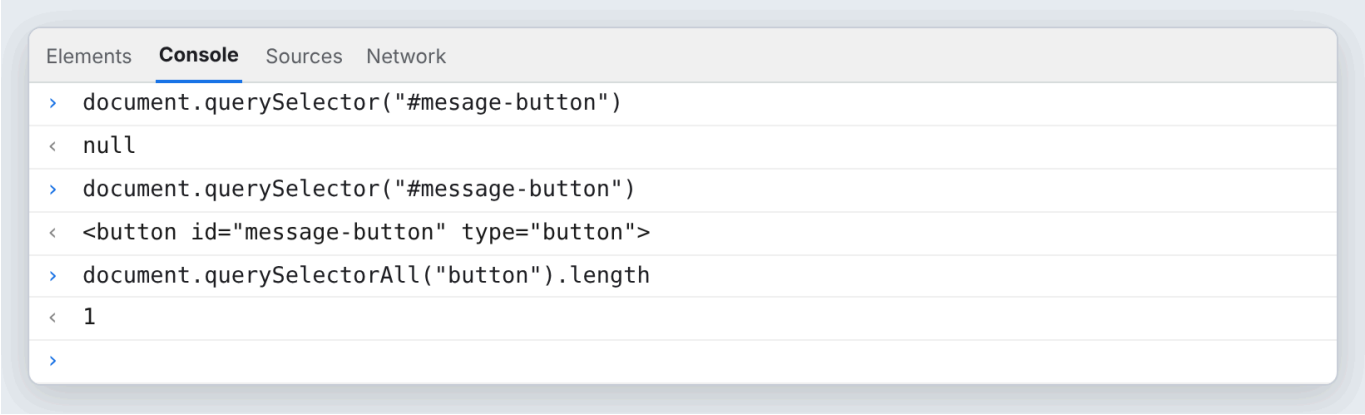
In your `practice-10/script.js` (or your own site's script), find one `querySelector` line. Write down the exact id or class it looks for. Now find that id or class in your HTML. Do they match exactly, including `#` or `.`?

5. How do you debug a selector?

You do not have to guess whether a selector works. You can ask the page.

© 2026 559 Solutions LLC

Open the page, open the Console, and type the `querySelector` call by itself. If the answer is `null`, the selector found nothing. If you see an element, the selector is fine and the bug is somewhere else. `document.querySelectorAll(...).length` tells you how many elements match, which is useful when you expected several.



One letter changes the answer from `null` to the button. Testing the selector by itself takes seconds and ends the guessing. (real output, rendered; evaluated in Chromium against the demo page)

On that same page, open the Console and type one of your `querySelector` calls. Is the answer an element or `null`?

6. How do you see what a variable holds?

Logic errors give no red line. To find them, you look inside the code while it runs.

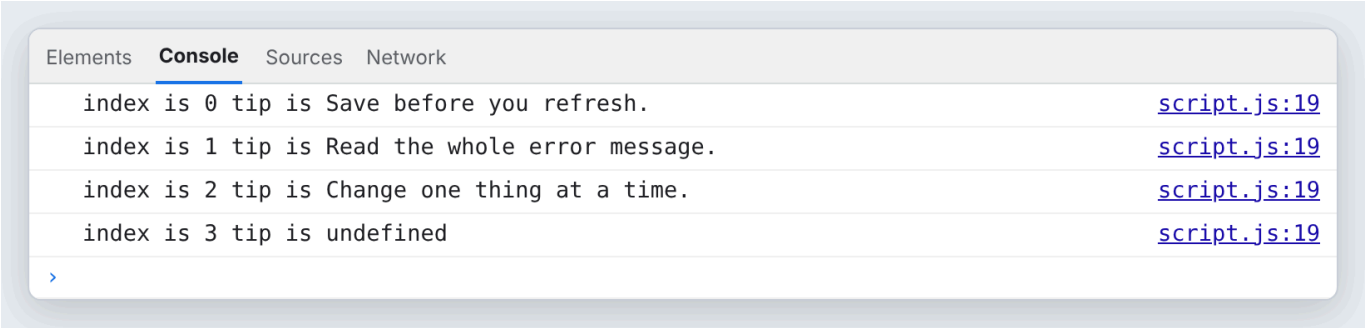
`console.log()`, in MDN's words, "outputs a message to the console." Put one line where the wrong thing happens, and print the values that line depends on. Each time that code runs, you get a new line in the Console, with the file and line number on the right. When you have found the bug, delete the log line.

In the practice page you will build in the Exercise, the Next button eventually shows an empty tip. Adding this as the first line inside the function that draws the tip:

```
console.log("index is", index, "tip is", tips[index]);
```

then loading the page and clicking Next three times gives this:

© 2026 559 Solutions LLC



The first line is the page load; the next three are the clicks. On the fourth line, `index` is 3, but there are only three tips, numbered 0 to 2. That is the bug, found in one run. (real output, rendered; captured in Chromium)

Question to answer before the Exercise: **what value does this variable hold** when `tips[index]` is `undefined`?

7. Why change one thing at a time?

When you find a suspect, it is tempting to fix three things at once.

If you change three things and the page works, you do not know which change fixed it. If it still fails, you do not know whether you fixed one bug and made another. Change **one** thing, save, and repeat the same test. Then you know exactly what each edit did. This is also why your Git commits in Module 12 hold one change each.

Look at the loop picture in concept 1 again: "Change one thing" comes before "Retest," every time.

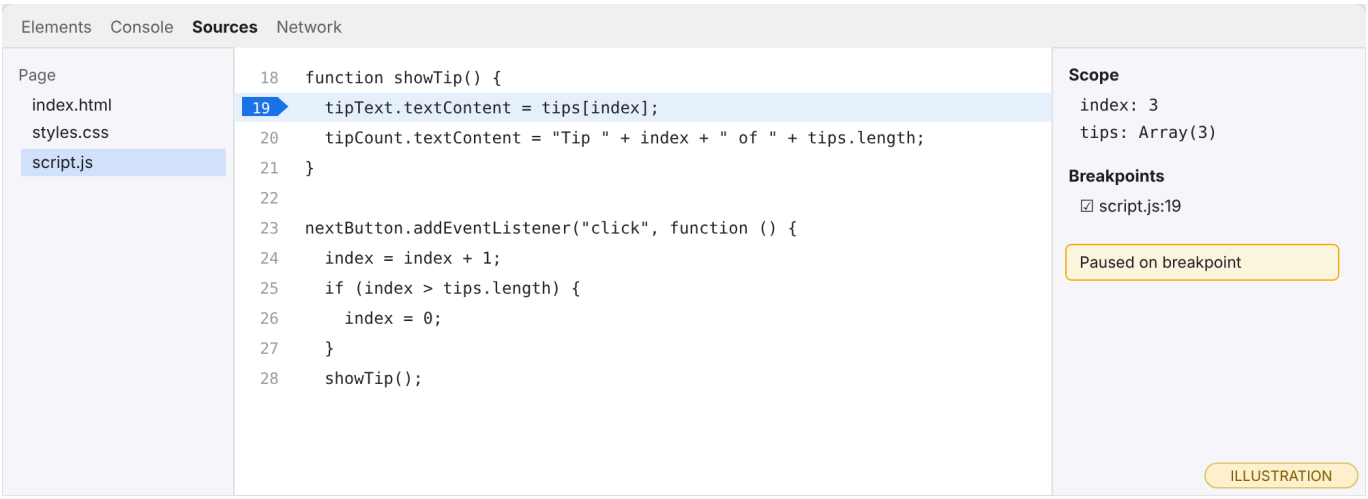
For the rest of this module, write down which single thing you are changing before you save.

8. What is the browser debugger?

DevTools has a **Sources** panel that can pause your code on a line, so you can look at every variable at that moment. You do not need it yet; `console.log()` is enough for this course. You should know it exists.

© 2026 559 Solutions LLC

Chrome's documentation (*Pause your code with breakpoints*) says to open the Sources panel, open the file, and click the line number to set a **line-of-code breakpoint**. The next time that line runs, the page freezes and the side panel shows the current values. Writing `debugger;` in your code does the same thing from inside the file, but only while DevTools is open. With DevTools closed, the line does nothing; MDN's *debugger* page says that without debugging tools "this statement has no effect."



A breakpoint on line 19 pauses the page so you can read `index` without a log line. The code is the real practice file; the panel layout and labels are drawn and simplified.

None today. When a `console.log()` would need five values at once, come back to this.

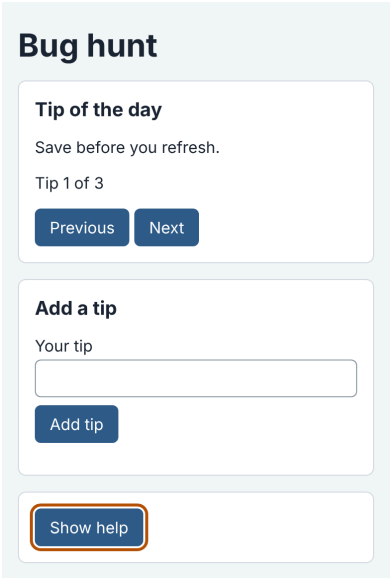
9. How do you test with the keyboard?

Some visitors never use a mouse. A page that only works with clicks is broken for them, and no red line will tell you.

MDN's *HTML: A good basis for accessibility* says to "try pressing the tab key" and watch focus move through the page. Press **Tab** to move forward and **Shift+Tab** to move back. Every link, button and form field should get a visible **focus state**, a ring or outline that shows where you are. A real

© 2026 559 Solutions LLC

`<button>` can be reached with Tab and pressed with Enter or Space; MDN notes you "lose the native keyboard accessibility" if you build a button out of a `<div>`. In CSS, `:focus-visible` is the selector for "focused, and the browser thinks the focus should be shown."



After five presses of Tab, focus is on "Show help," and the dark orange `:focus-visible` outline from the stylesheet shows it. (real: rendered in Chromium at 390 px, focus moved with five real Tab key presses)

Open your `practice-10` page (or your own site). Put the mouse away. Press Tab until you have visited every link and button. Write down any you could not reach or could not see.

10. What is the Action / Expected / Actual test matrix?

This is the testing format for the rest of the course. Every interactive feature gets one.

It is a table with three columns. **Action** is one thing a visitor does, written so someone else could repeat it. **Expected** is what should happen, written *before* you test. **Actual** is what did happen, written *after*. A row passes when the last two match. Here is a ready-to-use table for the tip card you will build:

© 2026 559 Solutions LLC

Action	Expected	Actual
Load the page	First tip shows; count reads "Tip 1 of 3"	
Click Next once	Second tip; "Tip 2 of 3"	
Click Previous on the first tip	Wraps to the last tip; "Tip 3 of 3"	
Click Next on the last tip	Wraps to the first tip; "Tip 1 of 3"	
Submit the form with the box empty	No tip added; a message asks for text	
Submit with only spaces	Same as empty	
Click Show help	The help text appears under it	
Tab through the page	Every button and the text box get a visible ring, in order	

The Actual column is empty on purpose. You fill it in during the Exercise.

Copy the empty table into your notes now. You will need it in ten minutes.

11. What is an edge case?

An **edge case** is an input or a moment at the boundary of what the code expects. Bugs live there.

Four edges come up in almost every feature in this course. **Empty input**: the visitor submits nothing, or only spaces. **The first item**: what does Previous do when you are already at the start? **The last item**: what does Next do at the end? **Wrap-around**: when you go past an end, do you land on the other end, or on nothing? Not every edge case finds a bug. You still test each one, because you do not know which one will.

Four of the eight rows in the table above are edge cases. Point to each one.

For one feature you built in Module 10 (or on your own site), write one edge case you have never tested.

Exercise

Your hands, about 60 minutes (estimate). Type it, do not paste it. This page has **six planted bugs**. Some print a red line. Some do not. Your job is to find all six with the Console, `console.log()`, the keyboard, and the test table.

1. **Make the files.** In `practice-11`, create a folder called `bug-hunt` with three files: `index.html`, `styles.css`, `script.js`.

© 2026 559 Solutions LLC

2. **Type `index.html`.**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Bug hunt</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main>
    <h1>Bug hunt</h1>
    <section class="card">
      <h2>Tip of the day</h2>
      <p id="tip-text"></p>
      <p id="tip-count"></p>
      <button id="prev-button" type="button">Previous</button>
      <button id="next-button" type="button">Next</button>
    </section>
    <section class="card">
      <h2>Add a tip</h2>
      <form id="tip-form">
        <label for="new-tip">Your tip</label>
        <input id="new-tip" type="text">
        <button type="submit">Add tip</button>
      </form>
      <p id="form-message" role="status"></p>
    </section>
    <section class="card">
      <div class="help-toggle" id="help-toggle">Show help</div>
      <p id="help" class="help">Use Previous and Next to move through the tips.
</p>
    </section>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

3. **Type `styles.css`.**

```
body { font-family: system-ui, sans-serif; margin: 0; background: #f4f6f8;
color: #1d2733; }
main { max-width: 640px; margin: 0 auto; padding: 16px; }
h1 { margin: 8px 0 16px; }
.card { background: #fff; border: 1px solid #d5dbe1; border-radius: 8px;
```


© 2026 559 Solutions LLC

```
padding: 16px; margin-bottom: 16px; }
.card h2 { margin-top: 0; font-size: 1.2rem; }
button, .help-toggle { font: inherit; padding: 8px 14px; border: 1px solid
#2f5d8a; border-radius: 6px; background: #2f5d8a; color: #fff; cursor: pointer;
display: inline-block; }
button:focus-visible, input:focus-visible { outline: 3px solid #b45309; outline-
offset: 2px; }
label { display: block; margin-bottom: 4px; }
input { font: inherit; padding: 8px; border: 1px solid #8a96a3; border-radius:
6px; width: 100%; box-sizing: border-box; margin-bottom: 8px; }
.help { display: none; }
.help.is-visible { display: block; }
```

4. **Type `script.js`.** Type it exactly as shown, bugs included. Line numbers matter, so do not add blank lines.

```
const tips = [
  "Save before you refresh.",
  "Read the whole error message.",
  "Change one thing at a time."
];
let index = 0;

const tipText = document.querySelector("#tip-txt");
const tipCount = document.querySelector("#tip-count");
const prevButton = document.querySelector("#prev-button");
const nextButton = document.querySelector("#next-button");
const tipForm = document.querySelector("#tip-form");
const newTip = document.querySelector("#new-tip");
const formMessage = document.querySelector("#form-message");
const helpToggle = document.querySelector("#help-toggle");
const help = document.querySelector("#help");

function showTip() {
  tipText.textContent = tips[index];
  tipCount.textContent = "Tip " + index + " of " + tips.length;
}

nextButton.addEventListener("click", function () {
  index = index + 1;
  if (index > tips.length) {
    index = 0;
  }
  showTip();
});
```


© 2026 559 Solutions LLC

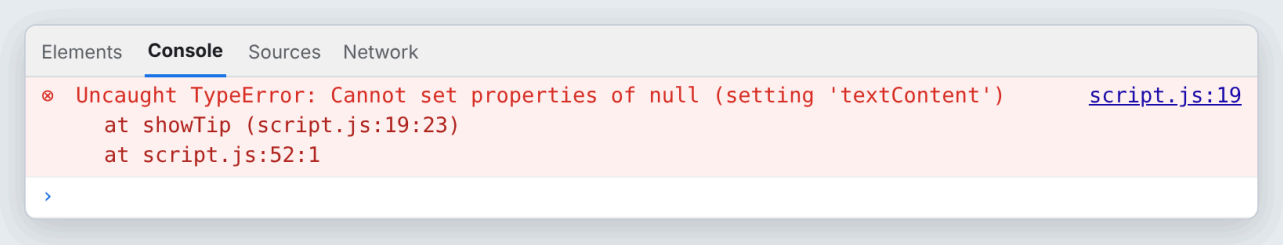
```
prevButton.addEventListener("click", function () {
  index = index - 1;
  if (index < 0) {
    index = tips.length - 1;
  }
  showTip();
});

tipForm.addEventListener("submit", function (event) {
  event.preventDefault();
  const text = newTip.value;
  tips.push(text);
  formMessage.textContent = "Added. There are now " + tips.length + " tips.";
  newTip.value = "";
  showTip();
});

helpToggle.addEventListener("click", function () {
  help.classList.toggle("visible");
});

showTip();
```

5. **Predict, then run.** Write your prediction: what will the tip card show? Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. The tip card has buttons but no tip and no count, and the Console shows this:

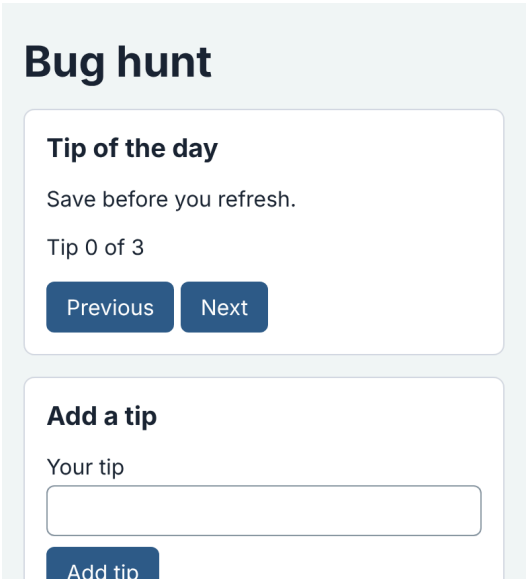


Read the stack from the top: the error happened inside `showTip` on line 19, and `showTip` was called from line 52. The real cause is on neither line. (real output, rendered; captured in Chromium loading the file. The Console frame is drawn.)

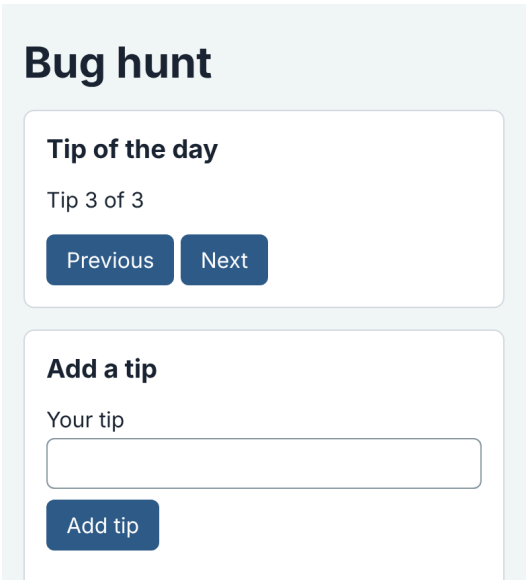
Use concepts 4 and 5 to find **bug 1**. Fix only that. Save. Press F5.

6. **Run the test table.** Now the page loads with no red line. Go through the Action / Expected / Actual table from concept 10, one row at a time, and fill in Actual. At first, every row fails; the count bug alone breaks the first four. Here are two of them:

© 2026 559 Solutions LLC



Row 1: Expected "Tip 1 of 3", Actual "Tip 0 of 3". No red line, so this is a logic error. (real: rendered in Chromium at 390 px)



After three clicks on Next the tip disappears. Concept 6 shows the `console.log()` line that explains why. (real: rendered in Chromium at 390 px, after three real clicks on Next)

7. **Fix one bug at a time.** For each row that failed: write down which line you think is wrong, change one thing, save, and run the **same row** again. Write the new Actual. Do not move on until the row passes.

8. **Break it on purpose.** Once Next wraps correctly, change `>=` (or whatever you used) back to `>`. Predict what row 4 will show now. Run row 4. Then fix it again and rerun row 4. Breaking a fixed bug and watching the test catch it is how you learn to trust the test.

© 2026 559 Solutions LLC

9. **Check the help button.** Click "Show help" with the mouse. Does the help paragraph appear? If not, compare the class name in `script.js` with the class names in `styles.css`. Fix it, save, press F5, and click again. Do this step before step 10: until the class name is fixed, the help text cannot appear, by mouse or by keyboard.
10. **Test the keyboard.** Put the mouse away. Press Tab from the top of the page. With the HTML as you typed it, focus goes Previous → Next → the text box → Add tip, and then leaves the page; "Show help" is never reached (checked in Chromium with real Tab presses). Fix it so Tab reaches "Show help" and Enter opens the help text. Hint: concept 9.
11. **Finish the table.** Every row's Actual now matches Expected. Add one more row of your own: one edge case the table does not cover yet.

▶ Check your hunt: the six bugs (open only after you have tried)

Common misconception

> *"The error is on the line the Console points to."*

What would you say? Hint: in the Exercise, the red line said 19, and the mistake was on line 8.

Summary

A bug is a gap between what you expected and what actually happened, so you write the expectation down first. A Console error gives you a type, a message, and a file and line number, and you read all three. `SyntaxError` means the code is not valid, `ReferenceError` means a name does not exist, and `TypeError` usually means a value is the wrong kind, most often `null`. `querySelector` returns `null` when it finds nothing, and the error appears later, where that `null` is used. You test a selector by typing it into the Console. `console.log()` shows what a variable holds at the moment the code runs. You change one thing at a time so you know what each change did. The Sources panel can pause code at a breakpoint when logs are not enough. A keyboard test with Tab and Shift+Tab finds problems no error message reports. The Action / Expected / Actual table is the course's test standard, and edge cases (empty input, first item, last item, wrap-around) are where most bugs hide.

Check your understanding

1. The Console says `Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')` at `script.js:14`. What do you check first, and is it on line 14?
2. Why can a page have a bug and show no red line at all? Give one example from the Exercise.

© 2026 559 Solutions LLC

3. **What would you test?** You add a fourth tip to the `tips` array. Which rows of the table would you run again, and why those?

Before next module

- [ ] You can name the three parts of a Console error and click through to the line
- [ ] You found all six bugs in `bug-hunt`, one change at a time, and broke one fix on purpose and caught it again
- [ ] Every row of your bug-hunt test table has an Actual that matches Expected
- [ ] You tabbed through a page you built (your `practice-10` page, or your own site) with the mouse put away, and wrote down what you could not reach
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *What went wrong? Troubleshooting JavaScript*
- MDN: *TypeError: "x" is (not) "y"* (the reference page for `null` and `undefined` errors)
- MDN: *Document: querySelector() method*
- MDN: *console: log() static method*
- MDN: *HTML: A good basis for accessibility* (the keyboard sections)
- MDN: *:focus-visible*
- Chrome for Developers: *Pause your code with breakpoints*
- freeCodeCamp: the free JavaScript Certification (checked 2026-10-11) has practice on this topic.

What to keep

Keep your filled-in bug-hunt test table and the empty Action / Expected / Actual template; you will use the template in every module from here on. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "What went wrong? Troubleshooting JavaScript"; MDN "TypeError: "x" is (not) "y""; MDN "Document: querySelector() method"; MDN "console: log() static method"; MDN "HTML: A good basis for accessibility"; MDN ":focus-visible"; Chrome for Developers "Pause your code with breakpoints". Checked 2026-10-11: Chrome for Developers "Open Chrome DevTools" (F12 and Ctrl+Shift+J); MDN "debugger"; MDN "function" (Hoisting).

PART 2

Project: A test plan for every interactive feature on your site

Your hands, about 2½ hours (estimate). In the Exercise you hunted six bugs in `practice-11` with a ready-made test table. Now you write the table yourself, for your real site. Every interactive feature gets its rows: the gallery, the FAQ, the Contact form and its states, the rating card, and the navigation and keyboard on every page. You run the plan on your computer first and fix what fails, one change at a time. In Part 3 you run it again on the live site and save it in your repository, so you can rerun it after every future change. You use only what Modules 1 to 11 taught:

- the Action / Expected / Actual table, with Expected written before you test (Core concept 10)
- the four edge cases: empty input, the first item, the last item and wrap-around (Core concept 11)
- the keyboard test: Tab, Shift+Tab, Enter, Space and the arrow keys, with a visible focus ring on every control (Core concept 9 and Module 4)
- reading a Console error: its type, its message, its file and line, and the `at` lines under it (Core concepts 2 to 4)
- `document.querySelector()` typed in the Console to test a selector, and properties such as `.alt`, `.ariaExpanded`, `.className` and `.textContent` to read a value (Core concept 5 and Module 9)
- one `console.log()` line to see what a variable holds, deleted afterward (Core concept 6)
- changing one thing at a time, and rerunning the same rows after each change (Core concept 7)
- device mode at 390 px (Module 3), and `pretendFail` from the Module 10 project
- one new kind of file, `test-plan.md`: plain text with `#` headings and tables made of `|` lines (Project step 3)

This project starts from the Module 10 packet's site: five pages (Home, About, Interests, Rate, Contact), one shared `styles.css`, and four page scripts: `gallery.js` on the Interests page, `faq.js` on the About page, `contact.js` on the Contact page and `rate.js` on the Rate page. Everything is committed and pushed, and the live site loads.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds one new file, `test-plan.md`, next to `index.html`. No other file changes, unless your plan finds a real bug in your site. Then the fix changes that file too.

The file. `test-plan.md` starts with a `#` heading, **My site test plan**, and one sentence that says what the file is. Then come five sections, each with a `##` heading and one table, in this order:

1. **Interests page: gallery**
2. **About page: FAQ**

© 2026 559 Solutions LLC

3. **Contact page: form**
4. **Rate page: rating card**
5. **Every page: navigation and keyboard**

Keep the "Every page" section last. In Module 12 you add rows to the end of the file, for a link that sits on every page.

The tables. Every table has the same three columns: **Action**, **Expected**, **Actual**. One row is one thing a visitor does.

- **Action** is written so someone else could repeat it without asking you: which page, what to click or press, and how many times.
- **Expected** is written before you test, from the brief of each feature (Modules 9 and 10), with the exact words the page shows.
- **Actual** is written after. A row that matches says `Pass`. A row that does not says what you saw instead, in a few words.

What the plan must cover. Every feature gets rows for the mouse **and** for the keyboard. Every one of the four edge cases appears wherever it applies: empty input on the Contact form and the rating card, the first and last item in the gallery and on the rating card, and wrap-around in both directions in the gallery. An edge-case row starts with `Edge case,` and names its edge, so you can find them at a glance. Each of the four feature sections has at least one row that reads a value in the Console, and the "Every page" section checks that no page has a red line.

Your site has no skip link. Module 4 explained the idea, but the packet projects did not add one, so the plan does not test one. If you added one yourself, give it a row in the last section.

Where the Actual column comes from. The finished file holds the Actual of your run on the live site, `https://your-username.github.io/my-site/` (Part 3). Before that, you run the whole plan on your computer, from disk, and fix what fails there first.

Here is the top of the sample's file, with its first section:

```
# My site test plan

Action / Expected / Actual for every interactive feature on my site. Rerun it after
every change.

## Interests page: gallery

| Action | Expected | Actual |
| --- | --- | --- |
| F5 on the Interests page, Console open | Photo 1 of 5, the first photo and its
caption; no red line in the Console | Pass |
| Click Next once | Photo 2 of 5, the second photo and its caption | Pass |
| Edge case, last item and wrap-around: click Next until Photo 5 of 5, then once more
```


© 2026 559 Solutions LLC

```
| Wraps to Photo 1 of 5, the first photo | Pass |
| Edge case, first item and wrap-around: F5, then click Previous once | Wraps to Photo
5 of 5, the last photo | Pass |
| After each click, type document.querySelector("#gallery-photo").alt in the Console |
The alt text describes the photo on screen | Pass: all five match |
| F5, Tab to Next, press Enter, then Space | Each key moves one photo and the counter;
Next has a dark orange ring | Pass |
```

The headings, the section order and the three column names must match the brief. The rows are yours: your own questions, messages and photos give you different Expected words. The sample's whole plan has 35 rows, and it is in Part 6, *Answers*.

The steps

Type every line. Do not paste: typing the rows makes you read each one.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`. Type `git status`. It should say `nothing to commit, working tree clean`: you start from what is already live.

2. **Find every feature, on paper.** Open each of the five pages in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. On each page, press Tab from the top to the bottom, and write down everything that responds to a click, a key or a submit.

   The sample's list had six groups: the five nav links on every page, the GitHub link on the Home page, the gallery's two buttons, the three FAQ questions, the Contact form with its four states, and the rating card. Then find your keyboard plan from Module 10, Project step 19. Its nine lines become rows of this plan.

3. **Make the file, then the gallery section.** In the Explorer, create `test-plan.md` next to `index.html`. The `.md` means Markdown: plain text that GitHub shows with formatting. You need four rules today, and Module 12 shows more:

   - A line that starts with `# ` is a heading. `## ` makes a smaller one.
   - A table is a block of lines that start and end with `|`. The `|` separate the columns. On a US keyboard, you type it with **Shift** and the key above **Enter**.
   - The first line holds the column names. The second line, `| --- | --- | --- |`, marks them as headings. Each line after it is one row.
   - Leave one empty line before each table, and none inside it. GitHub Docs (*Organizing information with tables*): "You must include a blank line before your table in order for it to correctly render."

© 2026 559 Solutions LLC

Type the heading, the sentence, and the gallery section with its first two rows. Leave Actual empty: the last cell is `| |`. **Write each Expected before you look at the page.** That is the whole point of the column.

```
# My site test plan

Action / Expected / Actual for every interactive feature on my site. Rerun it
after every change.

## Interests page: gallery

| Action | Expected | Actual |
| --- | --- | --- |
| F5 on the Interests page, Console open | Photo 1 of 5, the first photo and its
caption; no red line in the Console | |
| Click Next once | Photo 2 of 5, the second photo and its caption | |
```

```
<!-- Continue alone: four more rows for the gallery. The last item
     (Next on photo 5) and the first item (Previous on photo 1), each
     starting with "Edge case," and naming its edges. One row that reads the alt
text in
     the Console after each click. One keyboard row: Tab to Next,
     then Enter and Space. -->
```

To see your table as a table, press **Ctrl+Shift+V** in VS Code. VS Code's *Markdown and Visual Studio Code* page lists it as the preview shortcut. If a row shows as a plain line of `|` signs, see Problem 5 in Part 4, *Troubleshooting*.

4. **The FAQ section.** Under the gallery table, leave one empty line and type:

```
## About page: FAQ

| Action | Expected | Actual |
| --- | --- | --- |
| F5 on the About page, Console open | Three questions, each with +, no answer
showing; no red line | |
| Click question 1 | Its answer shows, the icon is −, the button is navy with
white text; questions 2 and 3 do not change | |
```

```
<!-- Continue alone: question 1 clicked again; an edge case with all
     three open at once; aria-expanded read in the Console with
     document.querySelector("#question-2").ariaExpanded, open and
     closed; and a keyboard row with Enter and Space on each question. -->
```


© 2026 559 Solutions LLC

If your keyboard gives a hyphen for the `-`, write "minus sign" instead.

5. **The Contact section.** The five paths from Module 10, Project step 9, are your first rows. Type the section with the first two:

```
## Contact page: form

| Action | Expected | Actual |
| --- | --- | --- |
| Edge case, empty: F5, press Send message with all three fields empty | Three
messages, focus in Your name, no reload | |
| Edge case, empty (spaces only): three spaces as the name, sam@ as the email,
hi as the message, Send | Three messages, focus in Your name | |
```

```
<!-- Continue alone: the bad email, the short message and the valid
     send (with your own thank-you words). Two edge cases at the limit:
     a message of 9 characters, and one of exactly 10. The form's class
     read in the Console during and after a send. The failed state with
     pretendFail = true. Then your keyboard lines from Module 10. -->
```

   Think this through: why test 9 and 10 characters, and not 3 and 50? A rule like "at least 10" breaks at its edge, if it breaks at all: `<` where `<=` was meant changes only what happens at exactly 10.

6. **The Rate section.** Type it alone, the same way: the heading, the two table lines, then your rows. Cover nothing chosen, a choice after the error, the first choice (1), the last choice (5), the hidden status line read in the Console, and the keyboard path from Module 10. The sample's first row was:

```
| Edge case, empty (nothing chosen): F5, press Send rating | "Please choose a
number from 1 to 5." under the numbers; focus (ring) on 1 | |
```

   This row is wrong on purpose: it promises a ring that the Module 10 brief does not. Keep it as it is, so you can see what a wrong Expected does to a run. Project step 12 comes back to it.

7. **The "Every page" section.** Type it alone, last in the file. It needs five rows: every nav link on every page, no red line on any page, the Tab order and rings on every page (light orange on the navy bar, dark orange elsewhere), the Home page's GitHub link by keyboard, and nothing scrolling sideways at 390 px. One row may cover all five pages; its Actual then says so.

8. **Check the plan before you run it.** Read every row as if a classmate had to run it alone. Could they repeat the Action? Could they say pass or fail from the Expected, without asking you? Here

© 2026 559 Solutions LLC

are three rows from a first draft. Rewrite each one in your notebook:

```
| Test the gallery | It works | |
| Click a question | The answer shows | |
| Type something wrong in the form and send | An error | |
```

None of the three can fail, so none of them tests anything. The sample's rewrites are in Part 6. Then count your edge-case rows: each of the four edges must be there.

9. **Plant one bug.** Real bugs often arrive this way: you tidy one file and forget the other. Open `interests.html` and change the counter's id from `photo-count` to `photo-counter`. Do not touch `gallery.js`. Save.

   Now try to forget it: the next step shows how much the plan finds alone.

10. **Run the whole plan from disk.** Open each page, press **F5**, keep the Console open, and run every row in order, top to bottom. Fill in Actual for each row as you go. Do not fix anything yet: first you want the whole picture.

    In the sample, 28 rows passed and seven failed. Five were in the gallery: the counter stayed at **Photo 1 of 5** while the photos changed, and the Console had a red line at load and another after every click. Only the alt-text row passed. The Rate page's first row failed: the message showed, but number 1 had no ring. And in the last section, "No red line on any page" failed on the Interests page.

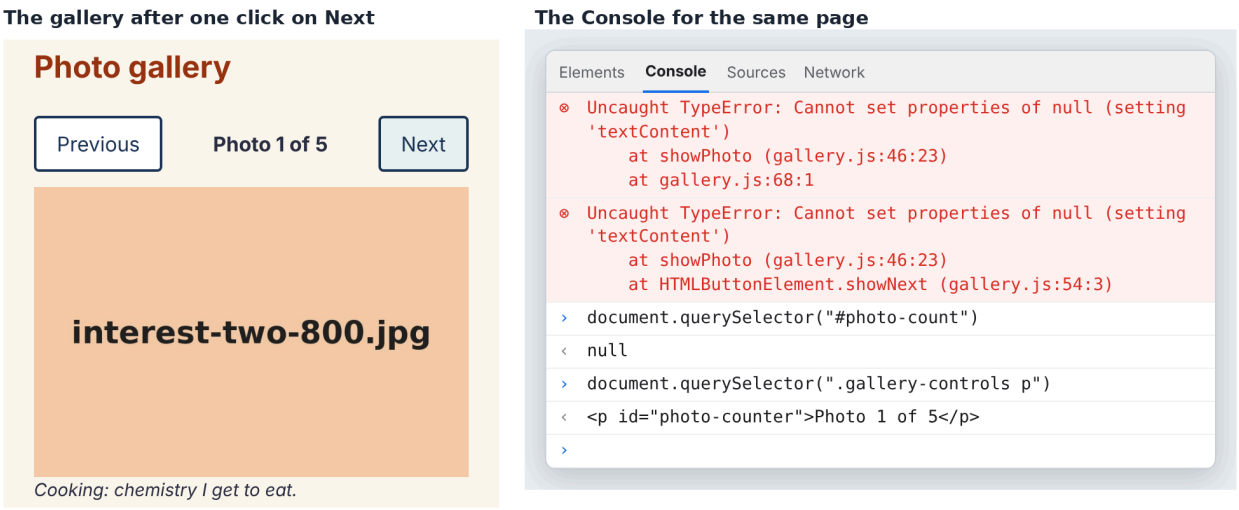
11. **Debug: the gallery.** Seven failures, but how many causes? Start with the first failing row, the one at load, and read its red line:

    ```
    Uncaught TypeError: Cannot set properties of null (setting 'textContent')
        at showPhoto (gallery.js:46:23)
        at gallery.js:68:1
    ```

    Read the `at` lines from the bottom up. Line 68 is `showPhoto();` in block 3, which runs once at load. It called `showPhoto`, and inside it, line 46 tried to set `textContent` on something that holds `null`. Line 46 is `counter.textContent = …`, so `counter` is `null`. A name gets its value in block 1, and `counter` gets it on line 6, from `document.querySelector("#photo-count")`.

    Now ask the page. In the Console, type `document.querySelector("#photo-count")`. The sample gave `null`: nothing on the page has that id. Then type `document.querySelector(".gallery-controls p")`, which finds the counter by where it sits. The sample gave `<p id="photo-counter">Photo 1 of 5</p>`. The selector and the HTML disagree.

© 2026 559 Solutions LLC



The planted bug, as the plan found it. The photo and caption moved on; the counter did not. The two lines at the bottom of the Console are the two checks above. Labels added above each capture. (real: rendered in Chromium at 390 px, and real output, rendered; captured in Chromium loading the files from disk. The Console frame is drawn.)

Want to see it from inside the function? Add `console.log("counter is", counter);` as the first line of `showPhoto`, save and press F5. The sample printed `counter is null` from `gallery.js:42`, right before the red line. The red line now said `gallery.js:47:23`, because the log line pushed every line below it down by one. Delete the log line.

Write down the one change you will make, then make it: put `photo-count` back in `interests.html`. The gallery brief from Module 9 names that id, so the HTML was wrong, not the script. Save, press F5, and run the first failing row again. It passed in the sample, with no red line. Then rerun the **whole** gallery section and the "No red line" row in the last section. All of them passed. Six failing rows, one cause, one change. The seventh failure, on the Rate page, has nothing to do with the gallery: it is the next step.

12. **A row that fails while the code is right.** Go back to the Rate row from step 6. The sample's Actual said: the message showed, focus went to 1, but number 1 had no ring. Before you change any code, find out who is wrong.

    Your Expected promised a ring. Reread the Module 10 brief: the card "puts focus on number 1", and "the dark orange ring shows on 1 only when you used the keyboard: after a mouse click, Chromium moves focus but draws no ring." Check both halves. F5, click Send rating with the mouse, then press the right arrow once: in the sample, 2 became the chosen number, so focus **was** on 1. Then run the keyboard row of the same section: with Tab and Enter, 1 got the dark orange ring.

    The code meets the brief; your Expected did not. So the one thing to change is the Expected. The sample's new Expected: `"Please choose a number from 1 to 5." under the numbers; focus on 1 (no ring after a mouse click; the right arrow then chooses 2)`. Rerun the

© 2026 559 Solutions LLC

row: it passed. A failing row says only that the page and your Expected disagree. The brief tells you which one is wrong.

13. **Break it on purpose.** In `contact.js`, in `findFirstProblem`, change `const name = nameInput.value.trim();` to `const name = nameInput.value;`. **Predict:** which rows of your Contact section will fail? Write down their numbers. Then save, press F5 and run the whole Contact section.

    In the sample, only the spaces-only edge case failed: no name message, the Email and Message messages, and focus in Email. With a good email and message, three spaces even "sent": **Thanks, . Your message passed every check.** One edge-case row stood between this change and your visitors. Put `.trim()` back and rerun the Contact section.

14. **Modify.** In `gallery.js`, change `let current = 0;` to `let current = 2;`. **Predict:** how many of your gallery rows will fail, and which? Write it down, then save and run the gallery section.

    The sample opened on **Photo 3 of 5**. Three rows failed: the load row, Next once (**Photo 4 of 5**), and Previous on the first item (**Photo 2 of 5**). The last-item row passed: from photo 5, Next still wrapped to photo 1. One change, three failing rows: that is why you rerun the whole section after any change. Change it back to `0` and rerun the section.

15. **Explain it.** Read `gallery.js` to yourself, out loud, from top to bottom. For each function, say which rows of your plan make it run. Do the same for `faq.js`, `contact.js` and `rate.js`; most people miss `finishSending` and its `if (pretendFail)` part. A function with no row is a gap in your plan: add a row.

16. **Test the plan.** Tick each box.

    - [ ] `test-plan.md` has the `#` heading, the sentence, and the five sections in the brief's order, "Every page" last
    - [ ] Every table has the columns Action, Expected and Actual, and shows as a table in the preview (Ctrl+Shift+V)
    - [ ] Every feature has mouse rows and a keyboard row, and each of the four feature sections reads at least one value in the Console
    - [ ] The four edge cases are all there, each row starting with `Edge case,` and naming its edges
    - [ ] A classmate could run every Action without asking you, and say pass or fail from every Expected
    - [ ] Every row has an Actual from your run from disk, and every row now passes
    - [ ] The planted id, the missing `.trim()`, `let current = 2;` and the `console.log` line are all undone
    - [ ] On every page, F5 with the Console open shows no red line

17. **Save it in Git.** Go to Part 3, *Finishing work in Git*. There you run the whole plan on the live site, fix anything that fails only there, and save the plan in your repository.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Your plan passes on your computer. Now you run it where your visitors are, on the live site, and save it in your repository with the same save routine as in Modules 1 to 10: `git status`, `git add .`, `git commit`, `git push`, then check the live site. Two things are new. You read `git status` as one more test: it tells you whether the code you are about to test live is the code you tested at home. And for the first time you run a whole written plan on the live site and record what it shows.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 10.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **Search for leftover test changes.** This project changed code on purpose three times, and added a log line once. In VS Code, open **Edit → Find in Files** (**Ctrl+Shift+F**) and search for `photo-counter`, then for `counter is`. Neither should be found in an `.html` or `.js` file. Then open `contact.js` and check that the name line in `findFirstProblem` ends in `.trim();`, and open `gallery.js` and check that it says `let current = 0;`.

3. **Read `git status` as a test (NEW).** Type:

   ```
   git status
   ```

   The sample printed:

   ```
   On branch main
   Your branch is up to date with 'origin/main'.

   Untracked files:
     (use "git add <file>..." to include in what will be committed)
           test-plan.md

   nothing added to commit but untracked files present (use "git add" to track)
   ```

   Read what is **not** there: no `modified:` line. Every page and script is exactly as you pushed it in Module 10, so the live site already runs the code your plan just passed at home. If a page or script shows as `modified:` and you did not fix a real bug in it, a test change is still in it: go back to step 2. If your plan did find a real bug and you fixed it, that file belongs here, and it goes into this commit with the plan.

4. **Run the whole plan on the live site (NEW).** Open `https://your-username.github.io/my-site/` with the Console open (**F12**, then the **Console** tab), and press F5. If a page acts like an old version, press **Ctrl+Shift+R** to reload without the browser's saved copy, as in Module 9. Then

© 2026 559 Solutions LLC

run every row, top to bottom, exactly as written. As you run each row, rewrite its Actual with what the live site did. When you are done, every Actual in the file comes from the live run.

Use a phone too if you have one, for the rows about 390 px and the Contact form. The steps in this packet were tested in Chromium with the files opened from disk. The live run could not be done when this packet was made, so this run is yours, and its Actual column is the record of it.

5. **Fix what fails live, one change at a time (NEW).** A row that passed at home and fails live usually means a file, not the code: a name whose capital letters differ, or a file that was never committed (Problems 1 and 2 in Part 4, *Troubleshooting*). For each failing row:

    - Write down the one change you think will fix it. Make it on your computer, save, and run the row from disk.
    - Run steps 6 to 8 now, with a message that names the fix, such as `Fix capital letters in gallery script name`. The plan goes into that commit as it is so far.
    - Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), press Ctrl+Shift+R, and run the row live again. Rewrite its Actual. Then carry on with the next row.

    If every row passed live, as it should when step 3 showed no `modified:` line, there is nothing to fix.

6. **Stage the plan.** Type:

    ```
    git add .
    ```

    It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Under **Changes to be committed** the sample showed one line: `new file: test-plan.md`.

7. **Commit.** Type:

    ```
    git commit -m "Add test plan for every interactive feature"
    ```

    The sample printed `1 file changed, 63 insertions(+)` and `create mode 100644 test-plan.md`. Each line of the file counts as one insertion, the empty lines too. Your commit id and line count will differ.

8. **Push.** Type:

    ```
    git push
    ```

    The last line ends in `main -> main`.

© 2026 559 Solutions LLC

9. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. The new commit is on top, with your Module 10 commit under it.

10. **See the plan on GitHub (NEW).** Open your repository on GitHub, `https://github.com/your-username/my-site`, and click `test-plan.md` in the file list. GitHub shows a Markdown file with its formatting, so your five sections should appear as headings with tables under them. If a section shows as lines full of `|` signs, see Problem 5 in Part 4. This could not be checked on GitHub when the packet was made; the table rules come from GitHub Docs (*Organizing information with tables*).

11. **Run one row again after the push.** The plan is not a page, so the push changed nothing your visitors see. Prove it: on the live site, run the "No red line on any page" row once more. From now on, every change to your site ends this way: rerun the rows that the change could touch, and when in doubt, the whole plan.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`, and `git log --oneline` shows `Add test plan for every interactive feature` on top. `test-plan.md` is in your repository on GitHub, and every Actual in it comes from your run on the live site. Every row passes.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. Problems 3 to 5 were made on purpose in the sample and run in Chromium 141, with the files opened from disk. Problems 1 and 2 happen on the live site, which could not be reached when this packet was made. For Problem 1, the sample served its files from a small web server on the same computer, to see what Chromium prints when a file is missing. After each fix, run the failing row again, then the rest of its section.

1. A row passes on your computer and fails on the live site

Symptom. On the live Interests page, the buttons do nothing, or the photo is a broken picture. On your computer, the same row passes. The live Console has a red line that starts with `Failed to load resource: the server responded with a status of 404`, with a file name beside it. The words in brackets after `404` depend on the server.

Why. 404 means the server has no file by that name. Your computer forgave something the live site does not:

What was wrong	How to see it
Capital letters differ, such as `Gallery.js` in the folder and `gallery.js` in the `<script>` line	Windows treats them as the same name; GitHub Pages does not (Module 1, Exercise step 16). In the sample, a server that tells capitals apart gave the 404 above for `gallery.js`.
The file was never committed	`git status` lists it under **Untracked files**
The file was committed but not pushed	`git status` says `Your branch is ahead of 'origin/main'`

Fix. Read the file name beside the red line and compare it with the Explorer, letter by letter. Change the name in the HTML to match the file, as Module 1 advised, rather than renaming the file. Then run the save routine and wait a few minutes before you test live again.

2. The live site still acts like the old version

Symptom. You pushed a fix, but the live row still fails the old way.

Why. Either GitHub Pages has not published the new version yet (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), or your browser shows a copy it saved earlier (Module 9).

Fix. Check that `git status` says your branch is up to date with `origin/main`. Wait a few minutes, then press **Ctrl+Shift+R** on the page to reload it without the saved copy, and run the row again.

© 2026 559 Solutions LLC

3. A Console check gives `Cannot read properties of null`

Symptom. You type a row's check, such as `document.querySelector("#gallery-photo").alt`, and the Console answers with a red line instead of a value:

```
Uncaught TypeError: Cannot read properties of null (reading 'alt')
```

Why. The selector found nothing on the page you are on, so `querySelector` gave `null`, and `null` has no `alt` (Core concept 4). In the sample, this happened in two ways: the check was typed on the About page, which has no gallery, and `document.querySelector("#question2").ariaExpanded` was typed without the dash in `question-2`. That gave the same message, ending in `(reading 'ariaExpanded')`.

Fix. Check which page the tab shows. Then type the selector alone, as in Core concept 5. If it gives `null`, compare it with the id in the HTML, letter by letter. The page's own script is not to blame: this error comes from the line you typed.

4. A row fails, but the code is right

Symptom. A row's Actual does not match, yet the feature seems to work. In the sample, the Rate page's first row expected a ring on number 1 after a mouse click on Send rating. The message showed, but no ring did (Project step 12).

Why. The Expected contradicted the brief. The Module 10 brief says focus goes to 1 and the ring shows only with the keyboard. Focus did move to 1. The ring rule uses `:focus-visible`, and Chromium did not show that ring after a mouse click. With Tab and Enter, the ring showed.

Fix. Before you change code, read the brief for that feature. If the page does what the brief says, change the Expected, and write why in a few words. If the page does not, change the code. Change only one of the two, then rerun the row.

5. The plan does not show as a table

Symptom. In the preview (**Ctrl+Shift+V**) or on GitHub, some rows show as a line full of `|` signs instead of a table row.

Why. A table has strict rules. In VS Code's preview, which is built on the markdown-it library (VS Code Docs, *Markdown and Visual Studio Code*), the sample broke a table three ways:

© 2026 559 Solutions LLC

What was wrong	What the preview showed
No `| --- | --- | --- |` line under the column names	No table at all; every line ran together as one paragraph
An empty line between two rows	The table stopped at the empty line; the rows after it showed as plain text
A `|` inside a cell, such as an Action with `||`	The cell was cut in two, and the columns shifted

There is a fourth rule for GitHub: "You must include a blank line before your table in order for it to correctly render" (GitHub Docs, *Organizing information with tables*). The VS Code preview showed a table even without that blank line, so check it by eye.

Fix. Give each table its `| --- |` line, no empty lines inside it, and one empty line before it. Write a `|` inside a cell as `\|`, or say it in words.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 12. If a line is a 1 or a 2, reread that Core concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can write down what I expect before I test, and describe a bug as the gap between expected and actual	☐	☐	☐	☐
I can read a Console error's type, message, file and line, and follow its at lines from the bottom up	☐	☐	☐	☐
I can tell SyntaxError, ReferenceError and TypeError apart, and say what each one usually means	☐	☐	☐	☐
I can trace "of null" back to the querySelector line that made the null, and test that selector in the Console	☐	☐	☐	☐
I can use console.log() to see what a variable holds when the code runs, delete it afterward, and say when a breakpoint would help more	☐	☐	☐	☐
I can change one thing at a time, and rerun the same rows after each change	☐	☐	☐	☐
I can test every page with the keyboard alone, and check that every control shows a focus ring	☐	☐	☐	☐
I can write an Action / Expected / Actual plan that someone else could run, and decide whether a failing row needs new code or a new Expected	☐	☐	☐	☐
I can find the edge cases in a feature (empty input, first item, last item, wrap-around) and give each one a row	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 10, seven rows failed, and Project step 11 fixed six of them with one change. Explain how one wrong id made five gallery rows and one "Every page" row fail, why the alt-text row still passed, and why you started with the first failing row and not with the last.

© 2026 559 Solutions LLC

2. Pick one row of your own plan that failed at least once: in the run from disk, in a break-it step, or on the live site. What did you write in Expected, what was the Actual, what was your first clue, and what one change made it pass?

3. A classmate's plan for a "Back to top" button has these rows. What is wrong with each one, how would you rewrite it, and what row is missing?

```
| Back to top | Works | |
| Click the button at the bottom | Page goes up | Pass |
| Scroll down and click Back to top | Expected: the top of the page shows | |
```


© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **`null` at line 14.** First check what the message says: the thing before `.addEventListener` on line 14 holds `null`. So the mistake is usually **not** on line 14. Line 14 is only where `null` was used. Find the `querySelector` line that gave that variable its value, usually higher up, in block 1. Compare its selector with the id or class in the HTML, letter by letter, including the `#` or `.`. Then type that `querySelector` call alone in the Console: `null` confirms it. Line 14 itself is wrong only if it uses the wrong variable name for an element that exists. (Core concepts 2, 4 and 5)
2. **A bug with no red line.** A red line appears only when JavaScript cannot go on. A logic error is code that runs from start to end and does the wrong thing, so there is nothing to stop and nothing to print. In the Exercise, the count said **Tip 0 of 3** on the first tip: `index` starts at 0, and the code showed it without adding 1. The class-name mismatch (`visible` against `.is-visible`) and the "Show help" `<div>` that Tab skips also printed nothing. Only a written Expected shows these bugs. (Core concepts 1 and 10)
3. **What would you test? A fourth tip.** Every row whose result depends on how many tips there are, and their Expected changes first: the load row now expects **Tip 1 of 4**, and the two wrap-around rows change most. Previous on the first tip should now land on the fourth tip, **Tip 4 of 4**, and Next on the last tip now starts from tip 4. The two form rows matter too, because the "Added" message counts the tips. The help and Tab rows do not touch the array, but a rerun of the whole table costs two minutes and catches what you did not think of. (Core concepts 10 and 11)

Common misconception

"The error is on the line the Console points to." The Console points to where JavaScript **stopped**, which is where a bad value was used. That is often not where the value was made. In the Exercise, the red line said `script.js:19`, `tipText.textContent = …`, but the mistake was the selector `#tip-txt` on line 8. Line 8 ran without complaint and stored `null`. Line 19 was the first to use it. MDN's troubleshooting page warns that "the error will not necessarily be on that line." So you read the location as a starting point, follow the `at` lines, and trace the bad value back to where it was made. (Core concepts 2 and 4)

Exercise checks

These are the results with the lesson's code, checked in Chromium 141 with the files opened from disk.

© 2026 559 Solutions LLC

- **Step 5.** Bug 1: `tipText` is `null`, because `#tip-txt` does not match `id="tip-text"`. `document.querySelector("#tip-txt")` gives `null` in the Console, and `document.querySelector("#tip-text")` gives the paragraph. Change the selector to `#tip-text`. After F5 the card shows **Save before you refresh.** and **Tip 0 of 3**, with no red line. (Core concepts 4 and 5)

- **Step 6.** Every row fails at first. Load: **Tip 0 of 3**. Next once: the second tip with **Tip 1 of 3**. Previous on the first tip: the third tip with **Tip 2 of 3**. Next on the last tip: no tip text and **Tip 3 of 3**. Empty submit: **Added. There are now 4 tips.** Spaces only: **Added. There are now 5 tips.** Show help: nothing. Tab: Previous, Next, the text box and Add tip get rings, then focus leaves the page; "Show help" is never reached.

- **Step 7.** One change per bug. Bug 2: `"Tip " + (index + 1) + " of "`. Bug 3: `if (index >= tips.length)`. Bug 4: trim the text and stop on an empty box, as in Module 10:

  ```
  const text = newTip.value.trim();
  if (text === "") {
    formMessage.textContent = "Please type a tip first.";
    return;
  }
  ```

  With these, rows 1 to 6 pass: **Tip 1 of 3**, **Tip 2 of 3**, **Tip 3 of 3** for Previous on the first tip, **Tip 1 of 3** for Next on the last tip, and **Please type a tip first.** for both bad submits, with the count still **Tip 1 of 3**. (Core concepts 6, 7 and 11)

- **Step 8.** With `>` back in place, row 4 shows no tip text and **Tip 4 of 3**: `index` reached 3, one past the last tip. Put `>=` back and row 4 passes again. (Core concept 11)

- **Step 9.** Bug 5: change `toggle("visible")` to `toggle("is-visible")`. A click on "Show help" then shows the help text, and a second click hides it. (Core concept 6)

- **Step 10.** Bug 6: make "Show help" a real button, keeping its id and class:

  ```
  <button type="button" class="help-toggle" id="help-toggle">Show help</button>
  ```

  Tab now goes Previous, Next, the text box, Add tip, Show help, each with the dark orange ring, and Enter on Show help opens the help text. (Core concept 9)

- **Step 11.** All eight rows pass. Good extra rows: a real tip, then Next until it shows (**Tip 4 of 4**); a tip with spaces around it, to check it is trimmed; Shift+Tab back through the page.

© 2026 559 Solutions LLC

Self-assessment questions

1. **One id, six failing rows.** The counter's id changed in the HTML, so line 6 of `gallery.js` stored `null` in `counter`. `showPhoto` sets the photo, the alt text and the caption first, and the counter last. So every call moved the photo and then stopped with a `TypeError` on line 46. That call happens at load and on every click, so every gallery row that checks the counter failed: load, Next once, both wrap-arounds and the keyboard row. The load error is also a red line on the Interests page, so the "No red line on any page" row failed. The alt row passed, because the alt is set on line 44, before the line that fails. You start with the first failing row because it is the simplest: no clicks, one red line, one stack to read. Its cause is often the cause of the rows after it, as here. (Project steps 10 and 11)
2. **Your own failing row.** There is no single answer. A good answer quotes the Expected as you wrote it before the test, gives the Actual in the words you wrote, names the first clue (a red line and its location, a value read in the Console, or what the page showed), and names exactly one change. It also says whether you changed code or the Expected, and why, as in Project step 12.
3. **The "Back to top" rows.** Row 1 has no repeatable Action and an Expected that cannot fail: rewrite as `Scroll to the bottom of the Home page, click Back to top` with `The page is at its top: the site name and Home link show`. Row 2 says "the button" and "goes up": which button, and up to where? It also already says `Pass` though nothing was written to compare it with. Row 3 puts the word `Expected:` into the cell and leaves Actual empty, which is fine, but "the top of the page" should name what you see there. The missing row is the keyboard: `Tab to Back to top, press Enter` with `Ring on the button; the page goes to the top`. An edge case is worth a row too: click it when you are already at the top. (Core concepts 9 to 11)

Project checks

- **Step 1.** `git status` says `nothing to commit, working tree clean`.
- **Step 3, continue alone.** The sample's other four gallery rows are in the whole plan below, from `Edge case, last item and wrap-around` to the keyboard row.
- **Steps 4 to 7, continue alone.** The sample's rows are in the whole plan below. Your messages, questions and thank-you words make your Expected different.
- **Step 8.** Three rewrites that can fail:

```
| Click Next once on the Interests page | Photo 2 of 5, the second photo and its
caption | |
| On the About page, click question 2 | Its answer shows, the icon is –, the
button is navy; questions 1 and 3 do not change | |
| Contact page: type sam@ as the email, fill the rest correctly, press Send
message | "That email looks incomplete. Check the @ and the part after it."
under Email; focus in Email | |
```


© 2026 559 Solutions LLC

- **Step 10.** 28 rows passed and seven failed: gallery rows 1 to 4 and 6, the Rate page's first row, and "No red line on any page". The sample's Actual for the gallery rows, before the fix: `Fail: Photo 1 of 5 and the first photo, but a red TypeError`, `Fail: second photo and caption, counter still Photo 1 of 5`, `Fail: counter never reaches 5; it stays at Photo 1 of 5`, `Fail: the books photo with Photo 1 of 5`, `Pass: all five match`, `Fail: the photo moves, the counter does not`.
- **Step 11.** `Uncaught TypeError: Cannot set properties of null (setting 'textContent')` at `showPhoto (gallery.js:46:23)` and `gallery.js:68:1` at load. After a click on Next, the second line is `HTMLButtonElement.showNext (gallery.js:54:3)`. `document.querySelector("#photo-count")` gives `null`. The log line prints `counter is null`. After the id is put back, all six rows that depended on it pass.
- **Step 12.** After a mouse click on Send rating, no ring on 1; the right arrow then chooses 2, with a ring. With Tab and Enter, the ring shows on 1. Only the Expected changes.
- **Step 13.** Only the spaces-only row fails: Email and Message messages, no name message, focus in Email.
- **Step 14. Photo 3 of 5** at load. Rows 1, 2 and 4 fail (**Photo 3 of 5**, **Photo 4 of 5**, **Photo 2 of 5**); the last-item, alt and keyboard rows pass.
- **Step 16.** If you followed the brief, every box is ticked. The most common misses: an Expected filled in after the test, and a section with no keyboard row.

Here is the sample's whole `test-plan.md`, 63 lines. Its Actual column comes from the run from disk in Chromium, after the fixes in Project steps 11 and 12. Yours comes from your run on the live site (Part 3, step 4).

```
# My site test plan

Action / Expected / Actual for every interactive feature on my site. Rerun it after
every change.

## Interests page: gallery

| Action | Expected | Actual |
| --- | --- | --- |
| F5 on the Interests page, Console open | Photo 1 of 5, the first photo and its
caption; no red line in the Console | Pass |
| Click Next once | Photo 2 of 5, the second photo and its caption | Pass |
| Edge case, last item and wrap-around: click Next until Photo 5 of 5, then once more
| Wraps to Photo 1 of 5, the first photo | Pass |
| Edge case, first item and wrap-around: F5, then click Previous once | Wraps to Photo
5 of 5, the last photo | Pass |
| After each click, type document.querySelector("#gallery-photo").alt in the Console |
```


© 2026 559 Solutions LLC

```
The alt text describes the photo on screen | Pass: all five match |
| F5, Tab to Next, press Enter, then Space | Each key moves one photo and the counter;
Next has a dark orange ring | Pass |

## About page: FAQ

| Action | Expected | Actual |
| --- | --- | --- |
| F5 on the About page, Console open | Three questions, each with +, no answer
showing; no red line | Pass |
| Click question 1 | Its answer shows, the icon is −, the button is navy with white
text; questions 2 and 3 do not change | Pass |
| Click question 1 again | Answer hidden, icon +, white button again | Pass |
| Edge case, all open: click all three questions | All three answers show at once |
Pass |
| Open question 2, then type document.querySelector("#question-2").ariaExpanded in the
Console | 'true'; after closing it, 'false' | 'true', then 'false' |
| F5, Tab to each question, press Enter, then Space | Enter opens it, Space closes it,
same as a click; ring on each | Pass |

## Contact page: form

| Action | Expected | Actual |
| --- | --- | --- |
| Edge case, empty: F5, press Send message with all three fields empty | Three
messages, focus in Your name, no reload | Pass |
| Edge case, empty (spaces only): three spaces as the name, sam@ as the email, hi as
the message, Send | Three messages, focus in Your name | Pass |
| Change the name to Sam, Send | Email and Message messages only, focus in Email |
Pass |
| Change the email to sam@example.com, Send | Message message only, focus in Message |
Pass |
| Edge case, one short: a message of 9 characters (123456789), Send | Message message,
focus in Message | Pass |
| Edge case, exactly enough: a message of 10 characters (1234567890), Send | No
messages; the form sends | Pass |
| Valid form (Sam, sam@example.com, Hello, I like your site.), Send | Button fades,
"One moment...", then after 1 second "Thanks, Sam. Your message passed every check."
on pale green; fields empty | Pass |
| During and after the send, type document.querySelector(".contact-form").className in
the Console | 'contact-form sending', then 'contact-form sent': one state at a time |
'contact-form sending', then 'contact-form sent' |
| Type pretendFail = true in the Console, then send a valid form | My failed message
on pale red; the words stay in the fields; F5 afterwards | Pass |
| Keyboard: Tab from Your name to Send message, Enter with all empty | Ring on each
field and the button; three messages, focus in Your name | Pass |
| Keyboard: with a name filled in, type sam@ in Email, press Enter | The form checks;
```


© 2026 559 Solutions LLC

```
focus stays in Email with its message | Pass |
| Keyboard: in Message, press Enter | A new line in the box, no check | Pass |

## Rate page: rating card

| Action | Expected | Actual |
| --- | --- | --- |
| Edge case, empty (nothing chosen): F5, press Send rating | "Please choose a number
from 1 to 5." under the numbers; focus on 1 (no ring after a mouse click; the right
arrow then chooses 2) | Pass |
| Then click 3 and press Send rating | The message is gone; the form is replaced by
"You chose 3 out of 5" | Pass |
| Edge case, first item: F5, choose 1, Send rating | "You chose 1 out of 5" | Pass |
| Edge case, last item: F5, choose 5, Send rating | "You chose 5 out of 5" | Pass |
| After a send, type document.querySelector("#rating-status").textContent in the
Console | 'Thanks for the feedback. You chose 5 out of 5.' (the number chosen) |
'Thanks for the feedback. You chose 5 out of 5.' |
| Keyboard: F5, Tab six times, Tab, Enter; then right arrow three times, Tab, Enter |
Ring on 1, then on Send rating; the message and focus on 1; then "You chose 4 out of
5" | Pass |

## Every page: navigation and keyboard

| Action | Expected | Actual |
| --- | --- | --- |
| On each of the five pages, click each nav link | The right page opens, and its own
nav link is bold | Pass on all five pages |
| F5 on each page, Console open | No red line on any page | Pass on all five pages |
| On each page, F5, Tab to the last link, button or field, then Shift+Tab back to Home
| Five nav links first with a light orange ring, then the page's links, buttons and
fields in reading order with a dark orange ring; Shift+Tab goes back the same way |
Pass on all five pages |
| Home page: Tab to the GitHub link, press Enter | My GitHub profile opens | Pass |
| Each page at 390 px (F12, then Ctrl+Shift+M) | Nothing scrolls sideways | Pass on
all five pages |
```

NEXT

Module 12: Git, GitHub and Publishing.
Free lesson: 559solutions.com/code559/module-12-git-github/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC